PAYMENT TRANSACTION MANAGEMENT SYSTEM

Project Documentation | MySQL 8.0+

1. Introduction

The Payment Transaction Management System is an extension of an Order Management System. It stores one payment record per order, records the payment mode and transaction amount, tracks payment status, and generates transaction and revenue reports.

Objectives

Store payment information linked to existing orders.

Restrict payment modes and statuses to valid values.

Prevent more than one payment record for the same order.

Retrieve payment history and identify failed or pending payments.

Analyze successful revenue and payment-method usage.

2. Payment Table Design

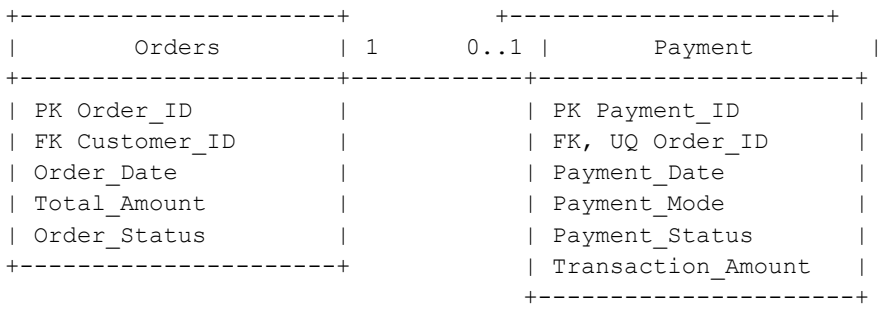
Column	Data Type	Constraint	Purpose
Payment_ID	INT	PRIMARY KEY	Unique identifier for each payment record.
Order_ID	INT	NOT NULL, UNIQUE, FOREIGN KEY	Links payment to an existing order and ensures at most one payment row per order.
Payment_Date	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Date and time of the payment attempt/record.
Payment_Mode	VARCHAR(30)	NOT NULL, CHECK	UPI, Credit Card, Debit Card, Net Banking, or Cash on Delivery.
Payment_Status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending', CHECK	Pending, Success, or Failed.
Transaction_Amount	DECIMAL(10,2)	NOT NULL, CHECK > 0	Amount recorded for the transaction.

Important: UNIQUE(Order_ID) guarantees at most one Payment row for each order. It does not guarantee that every order has a payment row. A foreign key prevents a payment from referencing an Order_ID that does not exist.

3. Updated ER Diagram

Relationship: Orders (1) —— (0..1) Payment

Each payment must belong to exactly one existing order. An order may have zero or one payment record. The relationship is implemented by Payment.Order_ID as a foreign key and UNIQUE column.



4. Sample Customer, Order and Payment Data

The SQL script creates demo Customers, Products, Orders, Order_Details, and Payment tables so it can run independently. If the Order Management tables already exist, use the existing tables and adapt the sample IDs to match them.

Sample Customers

Customer_ID	Customer_Name	Email
1	Arun Kumar	arun@example.com
2	Meena Ravi	meena@example.com
3	Kavin Raj	kavin@example.com
4	Divya S	divya@example.com

Sample Orders

Order_ID	Customer_ID	Total_Amount	Order_Status
1001	1	799.00	Delivered
1002	2	58999.00	Processing
1003	3	8999.00	Placed
1004	1	599.00	Placed
1005	4	9798.00	Processing
1006	2	1598.00	Placed

Sample Payments

Payment_ID	Order_ID	Mode	Status	Amount (₹)
5001	1001	UPI	Success	799.00
5002	1002	Credit Card	Success	58999.00
5003	1003	Debit Card	Failed	8999.00
5004	1004	Cash on Delivery	Pending	599.00
5005	1005	Net Banking	Success	9798.00

5006	1006	UPI	Pending	1598.00

5. Payment Transaction Operations

The SQL script includes queries to insert a new payment, display all payments, retrieve payment details for a selected order, update status after a retry, and delete an incorrect demo record when appropriate. The insert example is commented out because the referenced order must exist and must not already have a payment.

Status management

Success: the transaction has completed successfully.

Failed: the payment attempt was unsuccessful.

Pending: the result is not yet final or requires follow-up.

A retry can update the existing payment row instead of inserting a second row for the same order. In a production system, multiple attempts would normally be stored in a separate Payment_Attempt table; this assignment specifically requires at most one Payment row per order.

6. Payment Analysis and Revenue Reports

Transactions by payment mode: Groups records by Payment_Mode and counts how many transactions use each method.

Most preferred payment method: Returns the mode with the largest transaction count. Ties may return one of the tied modes because LIMIT 1 is used.

Total successful payment revenue: Sums Transaction_Amount only where Payment_Status = 'Success'; pending and failed amounts are excluded.

Revenue by payment method: Shows successful transaction count and successful revenue for each payment mode.

Average transaction amount: Calculates the average amount across all recorded payment statuses.

Customer payment history: Joins Customers, Orders, and Payment to show each customer's payment records.

Status totals: Counts Success, Failed, and Pending records, including a zero count for a status with no matching rows.

Orders with failed or pending payments: Lists orders that need payment follow-up.

7. Expected Results from Sample Data

Based on the six sample payment records, before running the optional retry update:

Successful transactions: 3; failed transactions: 1; pending transactions: 2.

Successful revenue: ₹69,596.00 (₹799.00 + ₹58,999.00 + ₹9,798.00).

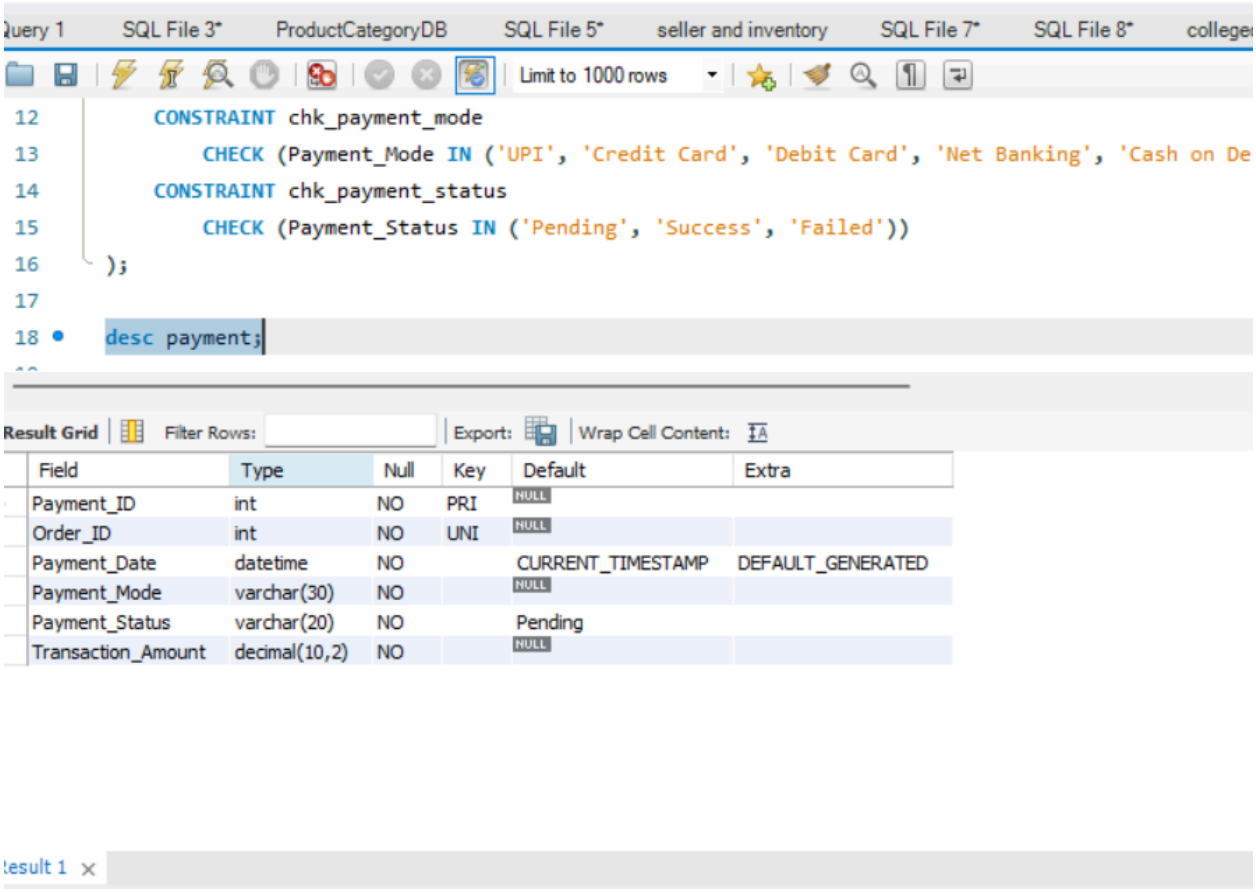
UPI has 2 transactions; each other payment mode has 1 transaction. UPI is the most used mode in this sample.

Orders 1003, 1004, and 1006 have failed or pending payment statuses.

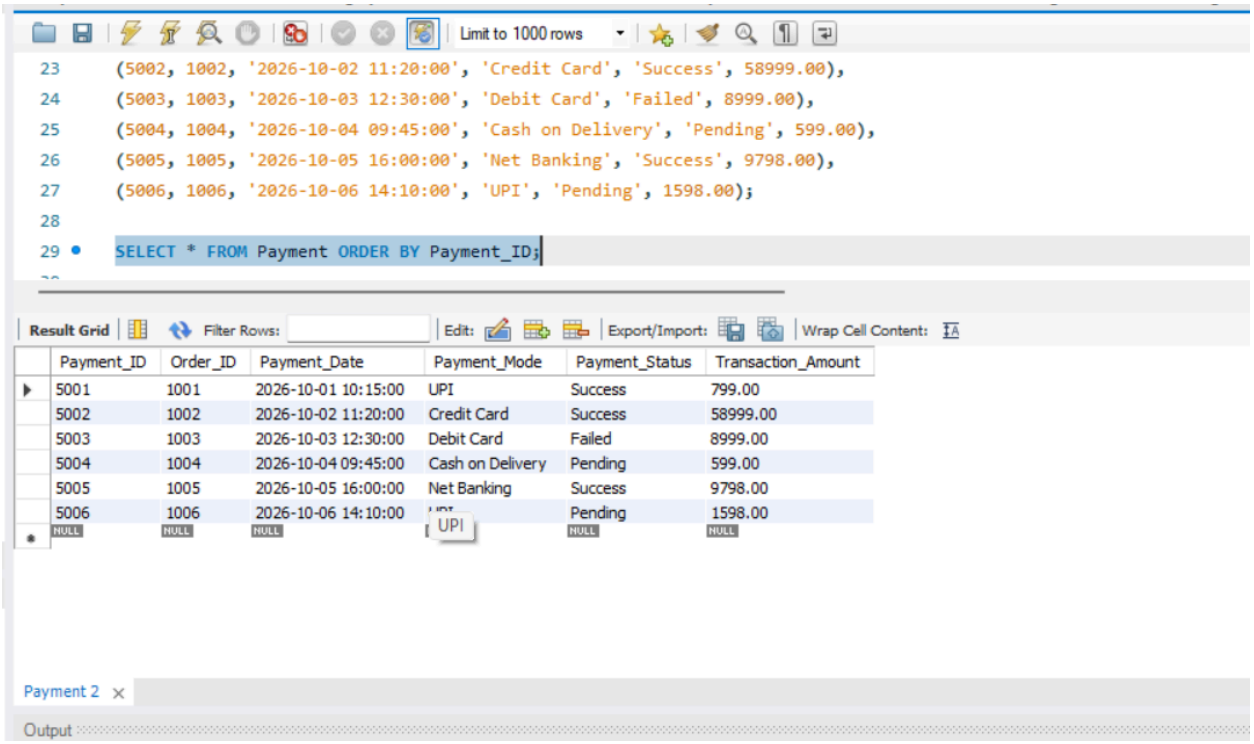
These are illustrative demo results, not real customer or payment information. If you execute status-update queries, the report totals will change accordingly.

8. Query Output Screenshots

1) Table Structure of Payment



2) Payment Rows

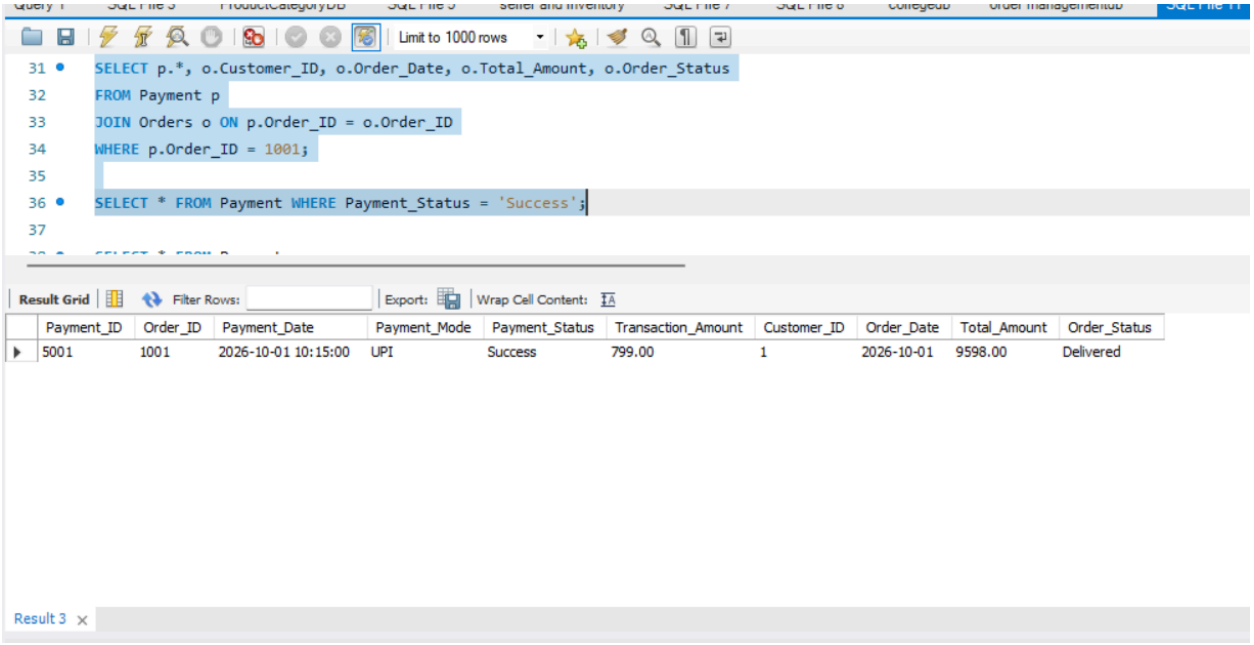


```
(5002, 1002, '2026-10-02 11:20:00', 'Credit Card', 'Success', 58999.00),
(5003, 1003, '2026-10-03 12:30:00', 'Debit Card', 'Failed', 8999.00),
(5004, 1004, '2026-10-04 09:45:00', 'Cash on Delivery', 'Pending', 599.00),
(5005, 1005, '2026-10-05 16:00:00', 'Net Banking', 'Success', 9798.00),
(5006, 1006, '2026-10-06 14:10:00', 'UPI', 'Pending', 1598.00);

SELECT * FROM Payment ORDER BY Payment_ID;
```

Payment_ID	Order_ID	Payment_Date	Payment_Mode	Payment_Status	Transaction_Amount
5001	1001	2026-10-01 10:15:00	UPI	Success	799.00
5002	1002	2026-10-02 11:20:00	Credit Card	Success	58999.00
5003	1003	2026-10-03 12:30:00	Debit Card	Failed	8999.00
5004	1004	2026-10-04 09:45:00	Cash on Delivery	Pending	599.00
5005	1005	2026-10-05 16:00:00	Net Banking	Success	9798.00
5006	1006	2026-10-06 14:10:00	UPI	Pending	1598.00

3) All Successful Payment

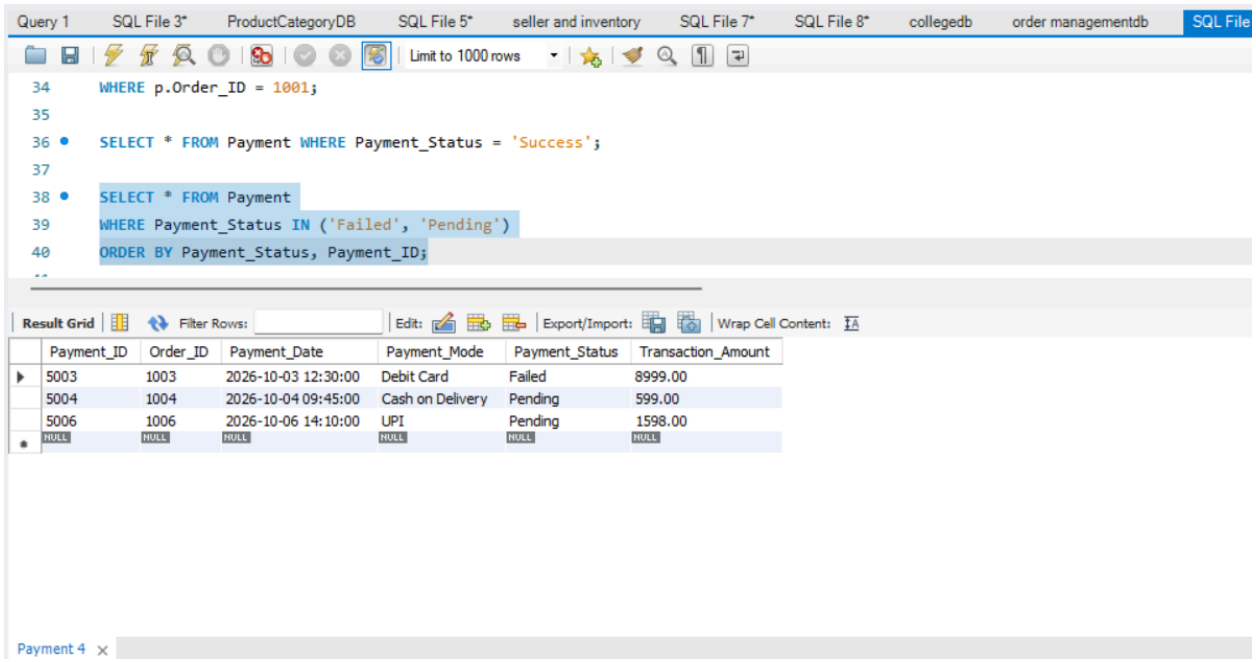


```
SELECT p.*, o.Customer_ID, o.Order_Date, o.Total_Amount, o.Order_Status
FROM Payment p
JOIN Orders o ON p.Order_ID = o.Order_ID
WHERE p.Order_ID = 1001;

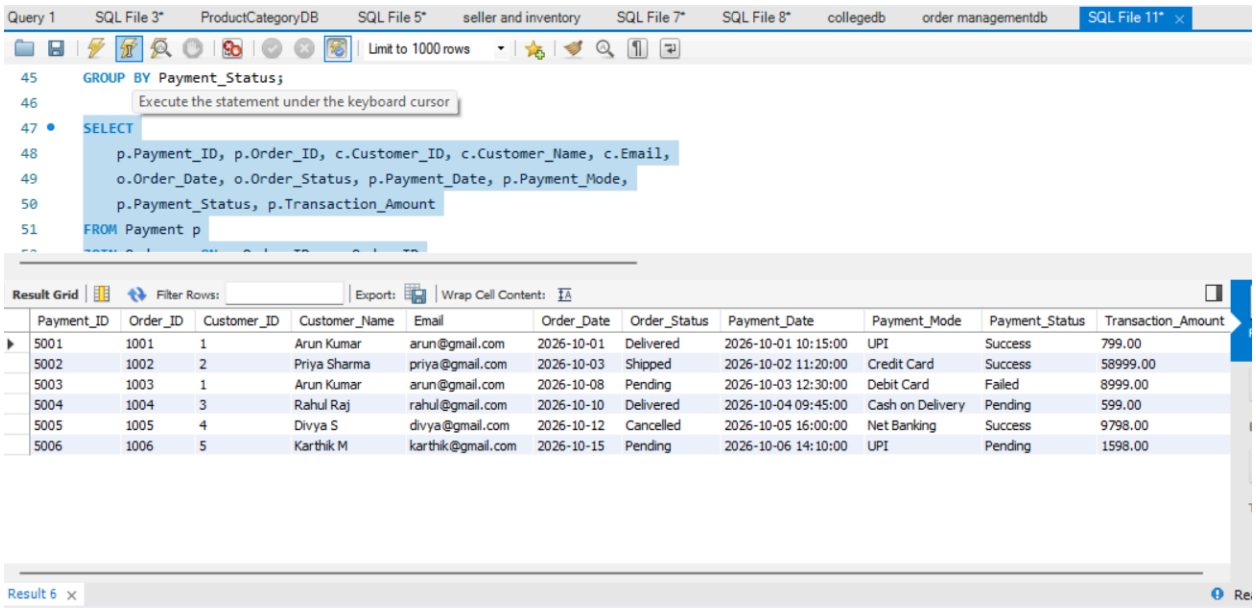
SELECT * FROM Payment WHERE Payment_Status = 'Success';
```

Payment_ID	Order_ID	Payment_Date	Payment_Mode	Payment_Status	Transaction_Amount	Customer_ID	Order_Date	Total_Amount	Order_Status
5001	1001	2026-10-01 10:15:00	UPI	Success	799.00	1	2026-10-01	9598.00	Delivered

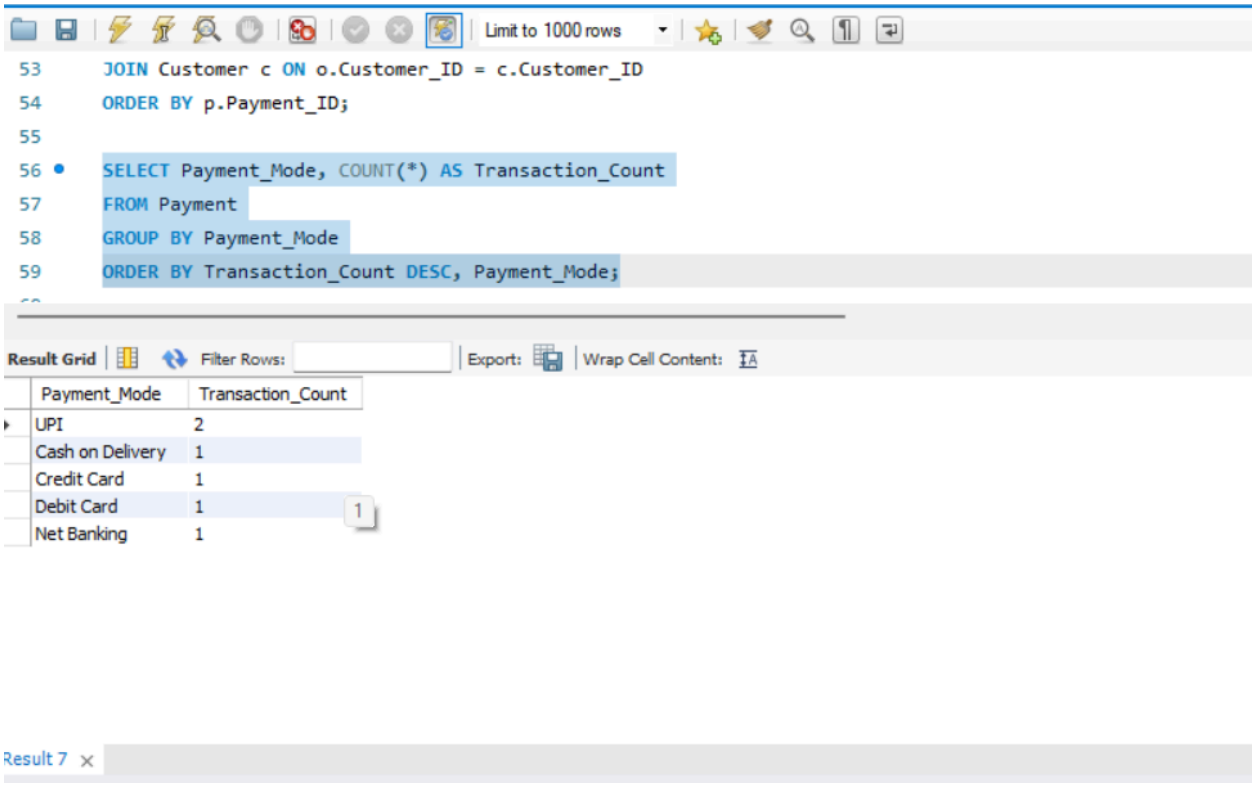
4) Failed or Pending Payments



5) Customer-order-payment Join



6) Transactions by Payment Mode



```
JOIN Customer c ON o.Customer_ID = c.Customer_ID
ORDER BY p.Payment_ID;

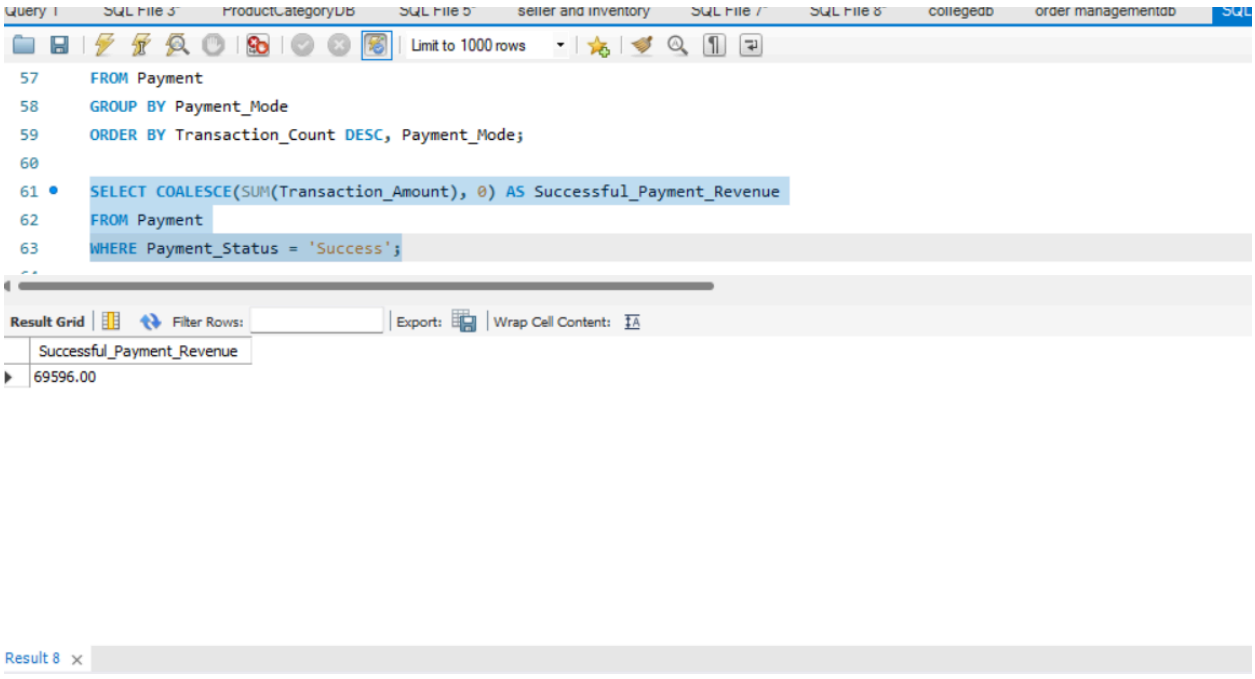
SELECT Payment_Mode, COUNT(*) AS Transaction_Count
FROM Payment
GROUP BY Payment_Mode
ORDER BY Transaction_Count DESC, Payment_Mode;
```

Result Grid

Payment_Mode	Transaction_Count
UPI	2
Cash on Delivery	1
Credit Card	1
Debit Card	1
Net Banking	1

Result 7

7) Successful Revenue



```
FROM Payment
GROUP BY Payment_Mode
ORDER BY Transaction_Count DESC, Payment_Mode;

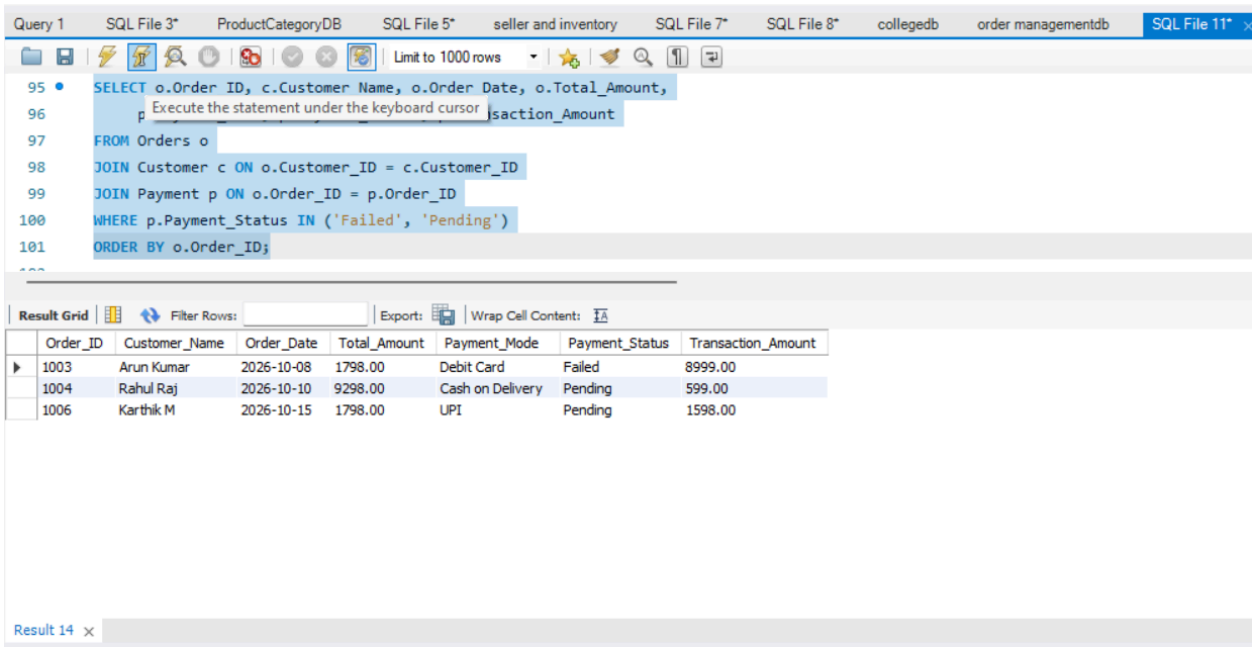
SELECT COALESCE(SUM(Transaction_Amount), 0) AS Successful_Payment_Revenue
FROM Payment
WHERE Payment_Status = 'Success';
```

Result Grid

Successful_Payment_Revenue
69596.00

Result 8

8) Order with failed or Pending Payments



9. Conclusion

The project demonstrates relational database design, primary and foreign keys, uniqueness and validation constraints, CRUD operations, status updates, joins, aggregate functions, and payment-revenue reporting. The unique foreign key relationship prevents duplicate payment records for an order while keeping payment records connected to the Order Management System.